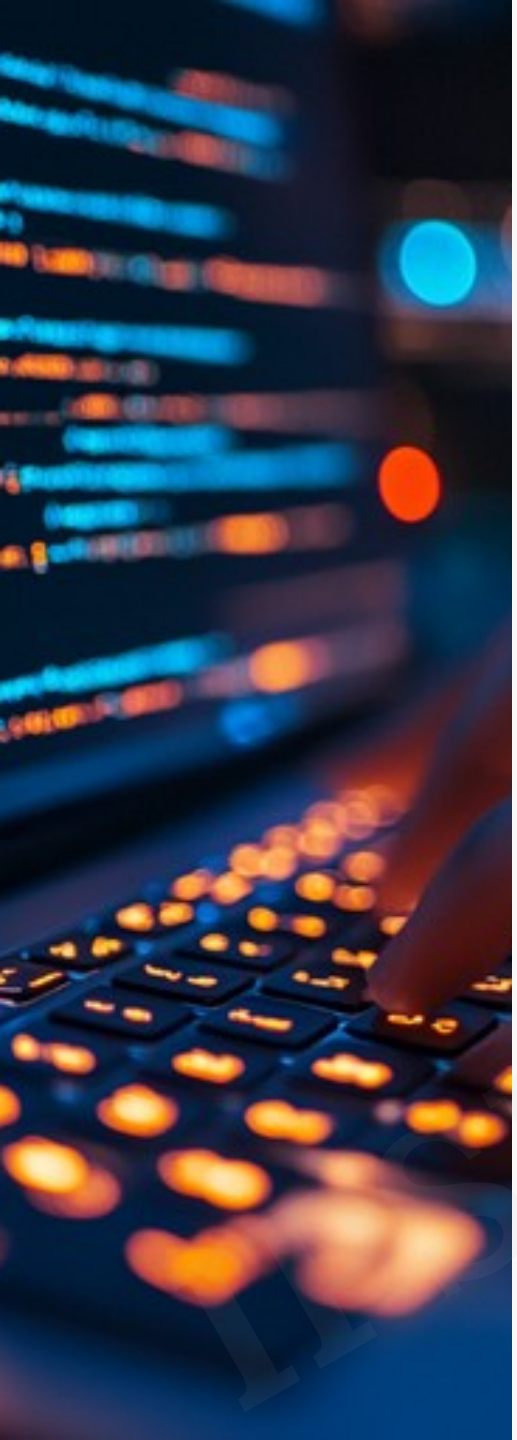


系統軟體

李建緯
雲科大資工系 智慧感知與系統整合實驗室
IPSI Lab, Yuntech-CSIE

IPSI Lab, Yuntech-CSIE

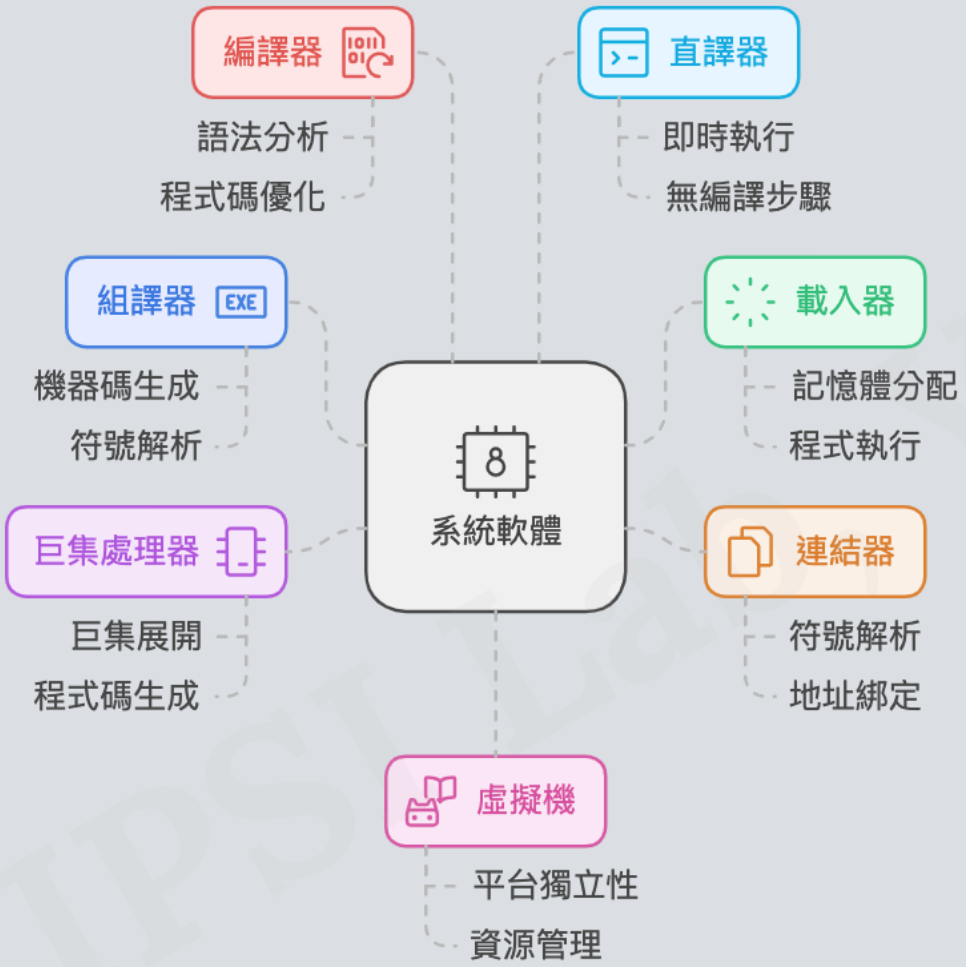
何謂系統程式？

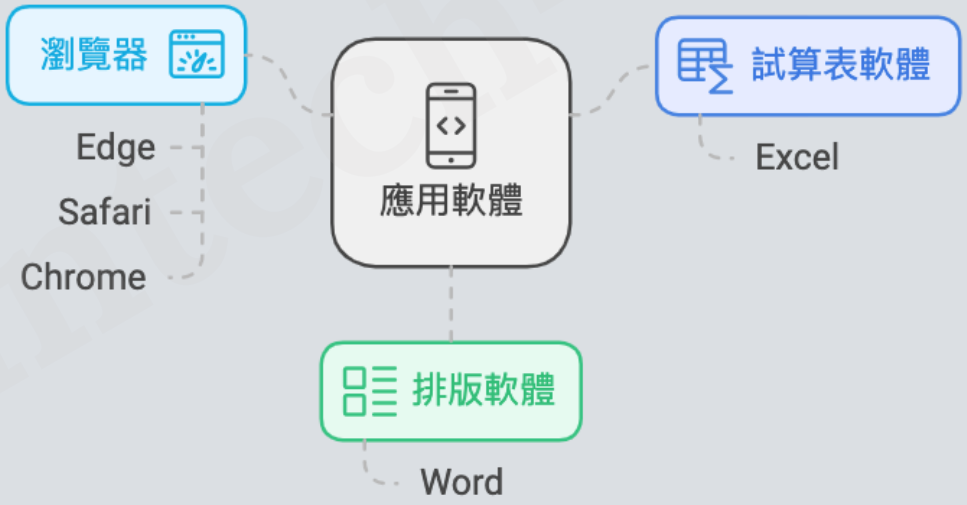
System Software (系統軟體)

- 設計給程式設計師使用的軟體, 就被稱為系統軟體
- 設計給一般大眾使用的軟體, 則稱為應用軟體。

IPSI Lab, Yuntech-CSIE

系統軟體 v.s 應用軟體


編譯器
語法分析
程式碼優化
直譯器
即時執行
無編譯步驟
組譯器
EXE
機器碼生成
符號解析
載入器
記憶體分配
程式執行
巨集處理器
巨集展開
程式碼生成
系統軟體
連結器
符號解析
地址綁定
虛擬機
平台獨立性
資源管理


瀏覽器
Edge
Safari
Chrome
應用軟體
試算表軟體
Excel
排版軟體
Word

系統軟體 vs. 系統程式 vs. 系統程式設計

系統軟體（System Software）

- 電腦的底層基礎設施與工具集合，用來管理硬體並提供基礎服務與介面。
- 典型成員包含作業系統核心（OS kernel）、裝置驅動（device drivers）、檔案系統（file system）、標準函式庫（standard library）、動態載入器（dynamic linker/loader）、Shell（command-line shell）、虛擬機 / 容器工具（virtual machine / container tooling），以及建置工具鏈的元件如編譯器（compiler）、組譯器（assembler）、連結器（linker）。

系統程式（System Program）

- 上述集合中的任何一個具體可執行成品或元件，例如 bash、ld.so、readelf、gcc、as、ld、systemd、strace。

系統程式設計（System Programming）

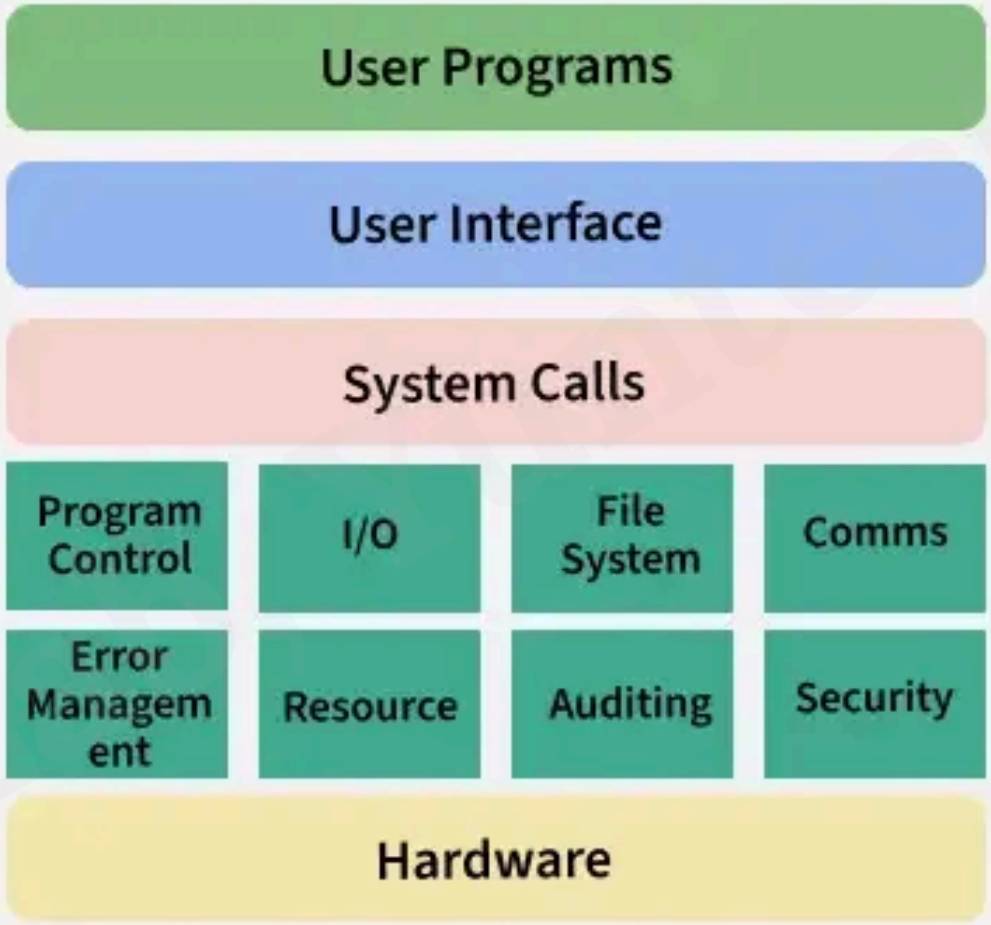
- 系統程式設計（System Programming）指的是一種貼近作業系統與硬體層的程式開發方法與活動。它和一般應用程式開發不同，重點在於直接使用作業系統提供的系統呼叫（system calls）與低階機制，來建構與維護系統層軟體。

作業系統相關的程式設計主題

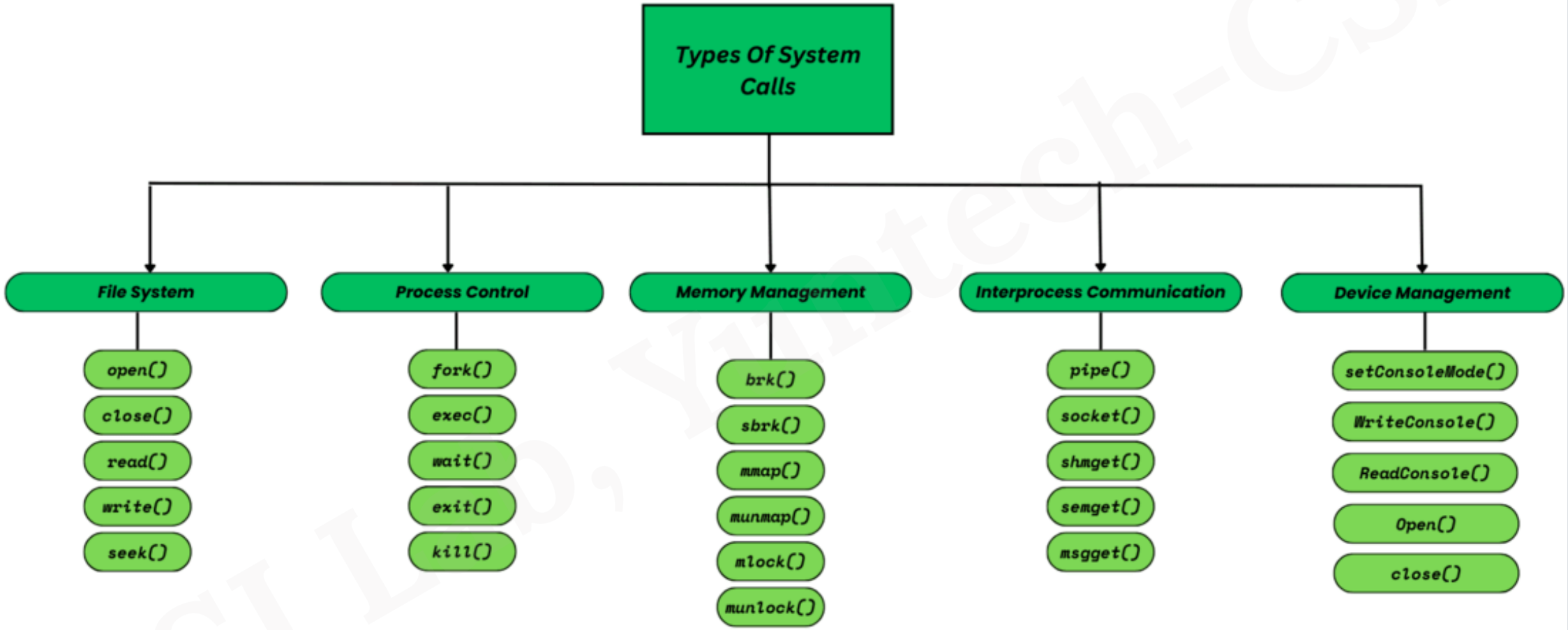
- 行程管理 (Process Management)
- 執行緒 (Thread)
- 行程通訊 (Process Communication)
- 並行控制 (Concurrency Control)
- 記憶體管理 (Memory Management)
- 檔案輸出入 (File Input/Output, File I/O)
- 驅動程式 (Device Driver)

IPSI Lab, Yuntech-CSIE

Introduction to System Call



Ref: https://www.geeksforgeeks.org/operating-systems/introduction-of-system-call/



Ref: https://www.geeksforgeeks.org/operating-systems/introduction-of-system-call/

系統軟體與系統程式

System Software & System Program

分類

- 建置工具鏈（Build Toolchain）
  - 編譯器（gcc/clang）、組譯器（as）、連結器（ld/lld）、建置工具（Make/CMake）
  - 功能：把原始碼翻譯成目的碼，進一步連結成可執行檔或動態函式庫
  - 關鍵：遵守 ABI（Application Binary Interface）（呼叫慣例、暫存器約定）、產生正確的目標格式（ELF）、處理重定位
- 執行期平台（Runtime Platform）
  - 作業系統核心、標準函式庫（libc）、動態載入器（ld.so）、驅動程式
  - 功能：提供系統呼叫與高層 API、管理資源、載入與連結執行期相依
  - 關鍵：系統呼叫介面、檔案系統、排程、記憶體管理、動態連結

相關聯之系統程式與軟體

1. **High-Level Language (HLL)**：高階語言（如 C、C++），是程式編寫的起點。
2. **Compiler**：編譯器，將高階語言轉換為機器碼或組合語言。如果產生的是組合語言，則需要通過組譯器進一步轉換；如果直接生成機器碼，則生成目標檔。
3. **Assembly Language**：組合語言，這是編譯器的可能輸出，接著需經由組譯器進行轉換。
4. **Assembler**：組譯器，將組合語言轉換為機器碼，生成目標檔。
5. **Object File (Object Code)**：目的檔(目的碼)，是編譯或組譯後生成的機器碼，但尚未完成連結。

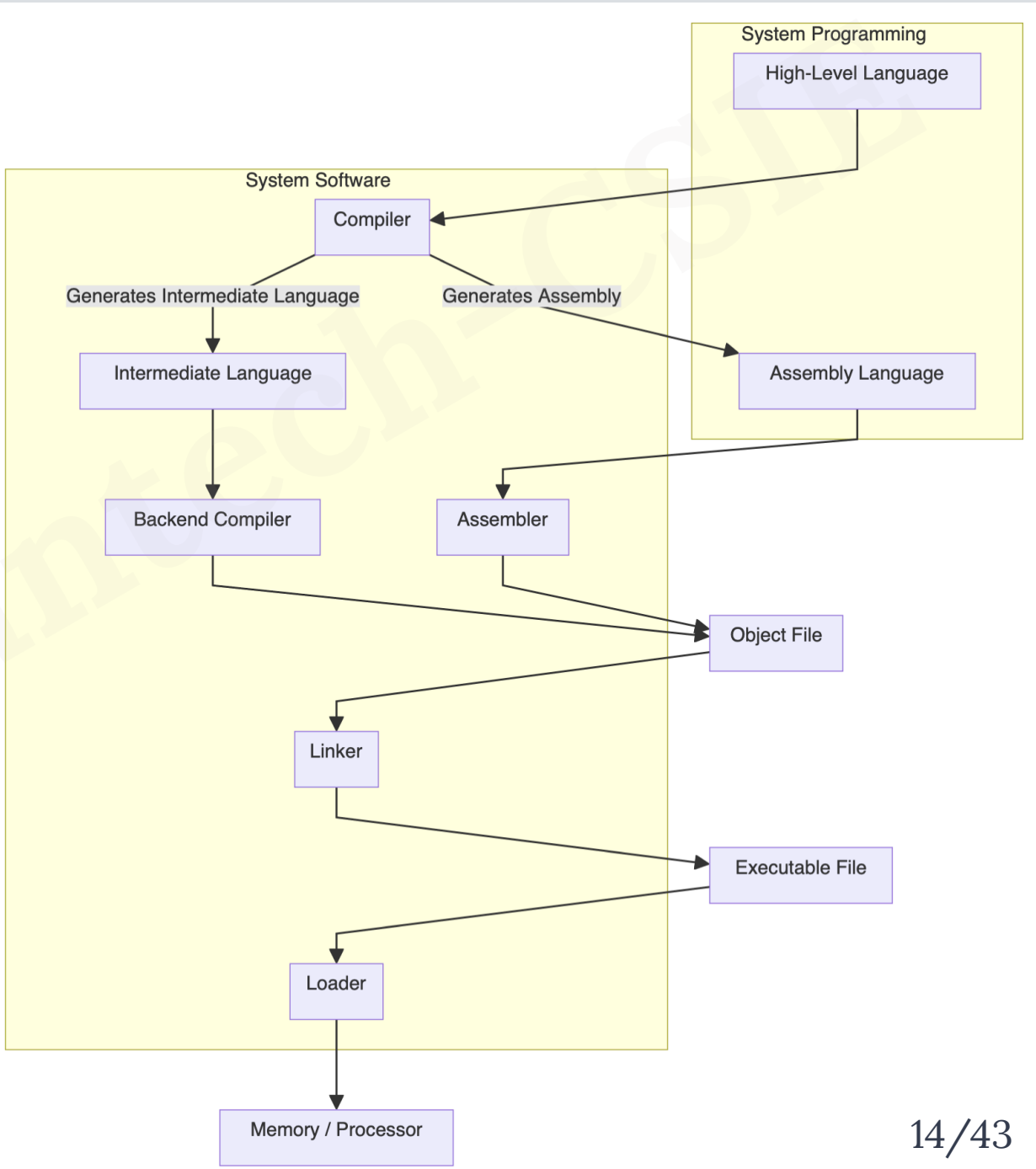
6. **Linker**：連結器，將多個目標檔連結起來，形成一個完整的可執行檔案。
7. **Executable File**：可執行檔，完成連結後的程式檔案，已經可以被載入並執行。
8. **Loader**：載入器，負責將可執行檔載入記憶體並分配資源，準備執行。
9. **Memory / Processor**：記憶體或處理器，程式最終執行的地方。

IPSI Lab, Yuntech-CSIE

執行路徑

執行路徑

- 高階語言 -> 編譯器 -> 組合語言
- 高階語言 -> 編譯器 -> 目的碼
- 組合語言 -> 組譯器 -> 目的碼
- 目的碼 -> 連結器 -> 可執行檔
- 可執行檔 -> 載入器 -> 記憶體，開始執行



從機器語言到組合語言

範例 1 ：2 進位串列形式的指令

```
00010011000100100011000000000000
```

範例 2 ：2 進位分組形式的指令

```
0001 0011 0001 0010 0011 0000 0000 0000
```

範例 3 ：16 進位分組形式的指令

```
13 12 30 00
```

範例 4 ：組合語言

```
ADD R1, R2, R3
```

範例 5 ：組合語言與機器指令的對應關係

```
ADD R1, R2, R3
13   1   2   3  000
```



各種組合語言的指令

```
指令 1 ：加法指令
        ADD R1, R2, R3
指令 2 ：減法指令
        SUB R1, R2, R3
指令 3 ：目標後置的指令寫法
        ADD R2, R3, R1
指令 4 ：以記憶體位址當參數
        LD R1, [300]
指令 5 ：直接存取記憶體
        ADD R1, R2, [300]
指令 6 ：使用變數
        ADD R1, R2, COUNT
指令 7 ：減少參數個數
        ADD R1, COUNT
指令 8 ：縮短指令長度
        ADD COUNT
```

從組合語言到高階語言

	C語言
指令 1 ：加法指令	
ADD R1, R2, R3	R1 = R2+R3
指令 2 ：減法指令	
SUB R1, R2, R3	R1 = R2 - R3
指令 3 ：目標後置的指令寫法	
ADD R2, R3, R1	R1 = R2+R3
指令 4 ：以記憶體位址當參數	
LD R1, [300]	R1 = [300]
指令 5 ：直接存取記憶體	
ADD R1, R2, [300]	R1 = R2 + [300]
指令 6 ：使用變數	
ADD R1, R2, COUNT	R1 = R2 + COUNT
指令 7 ：減少參數個數	
ADD R1, COUNT	R1 = R1 + COUNT
指令 8 ：縮短指令長度	
ADD COUNT	R0 = R0 + COUNT

[300] 代表位於記憶體 300 當中的值

累積器(Accumulator)是一種用來儲存運算結果的暫存器。
若累積器為R0

C 語言與組合語言的對應關係

C 語言	組合語言

`x = a + 3 * b - c*d;`	`LDI  R1, 3`
	`LD  R2, b`
	`MUL  R3, R1, R2`
	`LD  R1, c`
	`LD  R2, d`
	`MUL  R4, R1, R2`
	`LD  R1, a`
	`ADD  R2, R1, R3`
	`SUB  R2, R4, R2`
	`ST  R2, x`

IPSI Lab, Yuntech-CSIE

C 語言 (略)

- 1972年貝爾實驗室所發展出來的 C 語言
  - Ken Thompson與Dennis Ritchie (合稱 K&R)
  - K&R 用 C 語言設計出 UNIX 作業系統
    - C 語言特別適合用來撰寫作業系統等低階程式，因此是當今系統程式的主要語言
    - C 語言可以進行記憶體映射輸出入，這讓 C 語言特別適合用來撰寫嵌入式的系統程式

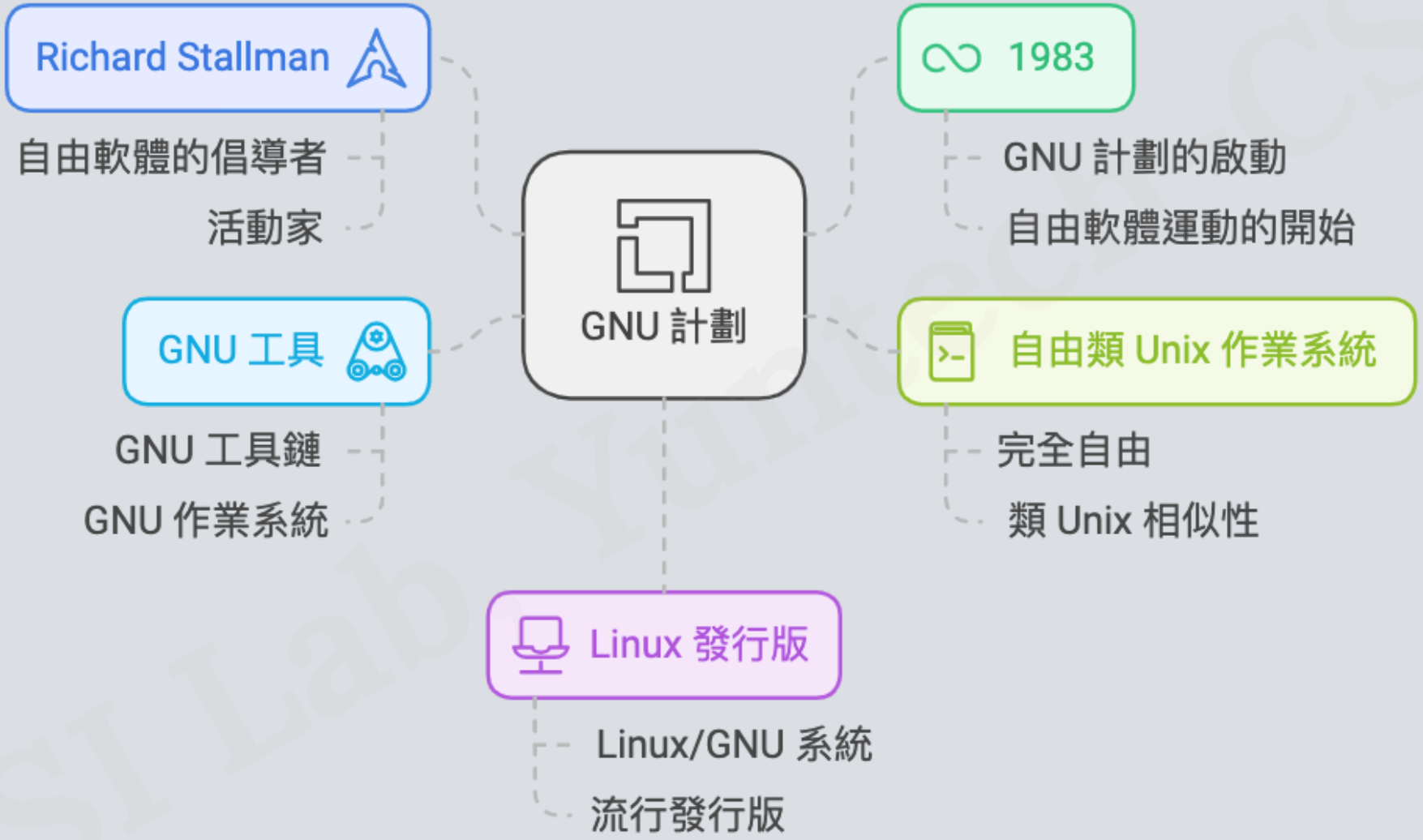
IPSI Lab, Yuntech-CSIE

GNU開發工具

GNU開發工具

- GNU 的全名是 GNU's Not Unix，這是一個遞迴縮寫（recursive acronym），其目的是表達 GNU 雖然與 Unix 系統相似，但它不是 Unix 系統。
- GNU 計劃由 Richard Stallman 於 1983 年發起，旨在創建一個完全自由的類 Unix 作業系統。這個計劃後來催生了許多自由軟體工具，其中最著名的是 GNU 工具鏈和 GNU 作業系統，並且成為了現代 Linux 發行版的重要組成部分（如 Linux/GNU 系統）。

IPSI Lab, Yuntech-CSIE


Richard Stallman
自由軟體的倡導者
活動家
1983
GNU 計劃的啟動
自由軟體運動的開始
GNU 計劃
GNU 工具
GNU 工具鏈
GNU 作業系統
自由類 Unix 作業系統
完全自由
類 Unix 相似性
Linux 發行版
Linux/GNU 系統
流行發行版

GCC 編譯器 (GNU Compiler Collection)

- **GCC（GNU Compiler Collection） 是一個開源的編譯器套件，支援多種程式語言和硬體架構。最初，GCC 主要是針對 C 語言設計的編譯器，但現在已擴展為支援多種語言如 C++、Fortran、Ada、Objective-C 和 Go。**
- **輸入**：
  - **原始碼檔案**：通常是 C 或 C++ 程式的原始碼檔案，副檔名為 `.c`（C 程式）、`.cpp`（C++ 程式）。
  - **其他檔案**：如 `.h` 檔案（標頭檔），這些檔案提供編譯時的額外定義。

- **輸出**：
  - **目的檔/目的碼**（Object File/ Object Code）：GCC 將原始碼編譯成機器碼，並輸出目的檔，副檔名通常為 `.o` 或 `.obj`。
  - **組合語言檔案**：當使用 `-S` 選項時，GCC 會輸出組合語言檔案，副檔名為 `.s`，這是編譯的中間結果。
  - **可執行檔案**：如果直接進行完整編譯和連結，GCC 也可以輸出最終的可執行檔，沒有固定副檔名（如 `a.out` 或自定義檔名）。
- **範例**：
  - `gcc -c main.c`：將 C 程式編譯成目的檔 `main.o`。
  - `gcc -o output main.c`：將 C 程式編譯並連結，生成可執行檔 `output`。

`as` 組譯器 (GNU Assembler)

- **`as` 是 GCC 工具鏈中的組譯器，用於將組合語言轉換為機器碼。當我們編寫組合語言程式或 GCC 編譯器生成組合語言時，`as` 組譯器負責將這些人類可讀的組合語言指令轉換為 CPU 可執行的二進位指令。**
- **輸入：**
  - **組合語言檔案：** `.s` 檔案，這是由 GCC 編譯器或手動編寫的組合語言程式。
- **輸出：**
  - **目的檔**（Object File）：組譯器會將組合語言檔案轉換成機器碼，並輸出 `.o` 或 `.obj` 副檔名的目的檔。

- **範例**：
  - `as program.s -o program.o`：將組合語言檔案 `program.s` 組譯成目的檔 `program.o`。

IPSI Lab, Yuntech-CSIE

`ld` 連結器 (GNU Linker)

- `ld` 是 GNU 工具鏈中的**連結器**，主要功能：
  - **符號解析**：連結器負責解決程式中使用的全域符號，例如跨檔案的函數呼叫或全域變數引用。
  - **目的檔合併**：`ld` 會將多個目的檔合併為一個可執行檔，並處理各檔案之間的位址分配和符號解析問題。
  - **函式庫連結**：`ld` 也能夠連結靜態或動態函式庫，將它們的內容合併到最終的可執行檔中。

IPSI Lab, Yuntech-CSIE

- **輸入**：
  - **目的檔**（Object Files）：由 GCC 或 `as` 生成的 `.o` 檔案。
  - **靜態函式庫**（Static Libraries）：如果程式依賴於外部靜態函式庫（如 `.a` 檔案），`ld` 也會將這些檔案作為輸入。
  - **動態函式庫**（Shared Libraries）：當程式需要動態連結庫（如 `.so` 檔案），`ld` 也會將它們載入連結過程。
- **輸出**：
  - **可執行檔案**：`ld` 會將目的檔連結成一個可執行檔案，該檔案沒有固定副檔名（如 `a.out`，或者用戶指定名稱）。
  - **動態函式庫**：當需要生成動態函式庫時，`ld` 會輸出 `.so` 副檔名的檔案。
- **範例**：
  - `ld file1.o file2.o -o output`：將兩個目的檔 `file1.o` 和 `file2.o` 連結成一個可執行檔 `output`。

Make 專案建置工具 (GNU Make)

- **GNU Make 是一個專案建置工具，用來自動化和管理程式編譯和建置過程。它根據一個稱為 `Makefile` 的配置文件，定義專案的編譯規則和依賴關係，並確保程式的各個部分按照正確順序進行編譯。**
- **輸入：**
  - **`Makefile`**：`Makefile` 是 Make 的配置文件，定義了專案的編譯規則、檔案依賴性以及需要執行的命令。`Makefile` 可以指定各個源檔、目的檔和最終生成的可執行檔之間的依賴關係。
  - **原始碼檔案**：`Makefile` 通常會引用專案中的源檔案（如 `.c`、`.cpp`），並根據依賴關係來觸發 GCC 或其他工具進行編譯。

- **輸出**：
  - **目的檔和可執行檔**：`make` 根據 `Makefile` 的定義，最終會輸出編譯好的目的檔或可執行檔案。這些檔案的類型取決於 `Makefile` 的規則。
  - **中間檔案**：在大型專案中，`make` 可能會生成一系列中間的目的檔，這些檔案可以被後續步驟用於最終連結。
- **範例**：
  - 在專案目錄中執行 `make`，`Makefile` 會控制整個專案的編譯過程，最終輸出可執行檔案或目的檔。
  - `make clean`：這是一個常見的命令，會執行 `Makefile` 中定義的「清理」指令，刪除所有編譯生成的中間檔案和目的檔。

IPSI Lab, Yuntech-CSIE

標準函式庫 (`libc.a`)

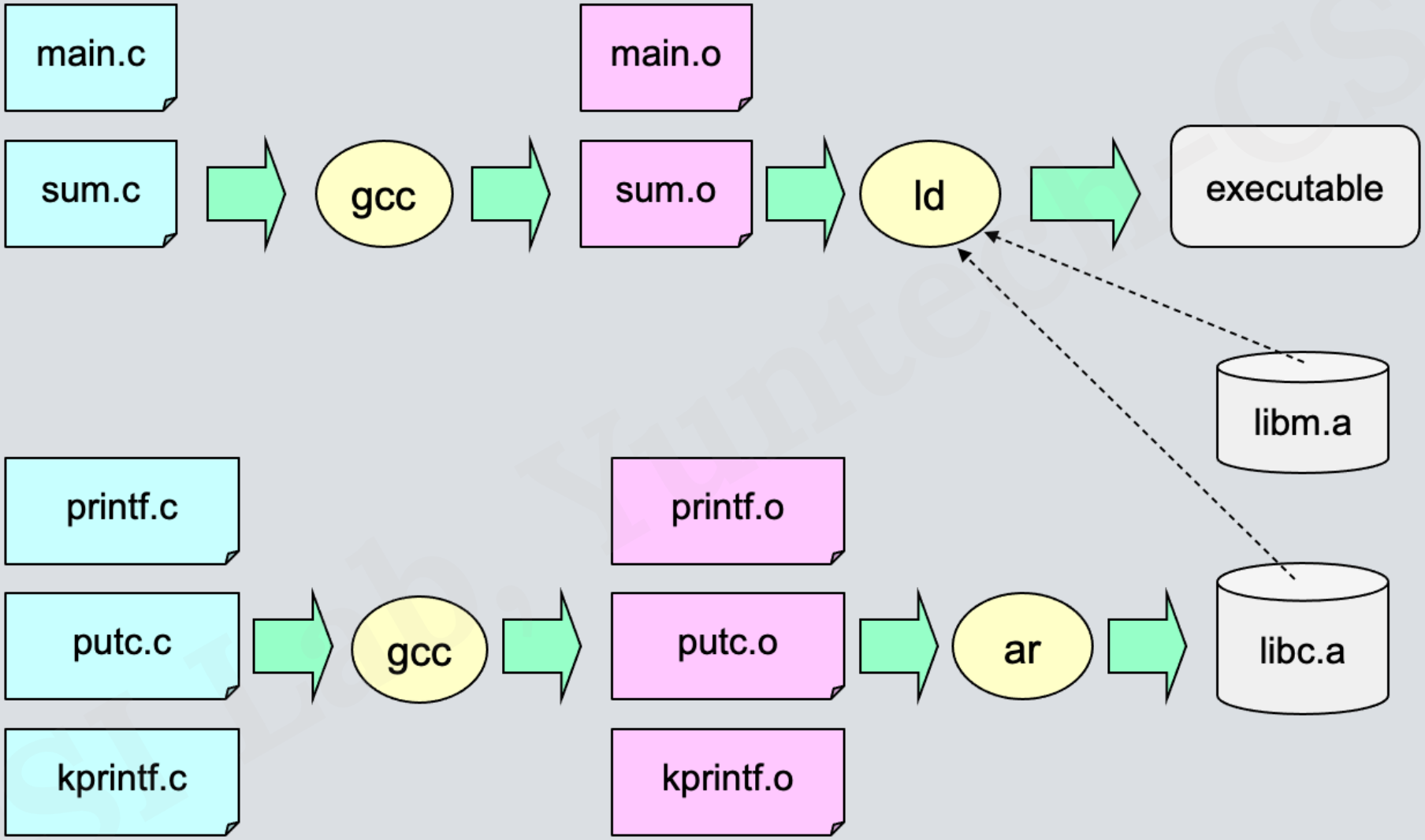
- **函式庫構建**：`printf.c`、`putc.c` 和 `kprintf.c` 等常用函式的編譯過程。這些函式會被編譯成目的檔，然後通過**函式庫工具** `ar` 工具將它們打包成靜態函式庫 (`libc.a`)。
- **靜態函式庫**：當程式需要使用 `printf` 等標準輸出函式時，無需每次重複編譯函式，只需引用預先建立好的靜態函式庫。

IPSI Lab, Yuntech-CSIE

使用GNU開發工具

GNU開發工具整體工作流程

1. `gcc` 編譯器將高階語言（如 C、C++）的源程式編譯成 **目的檔**（`.o` 檔案）。
2. `as` 組譯器將組合語言（`.s` 檔案）轉換為 **目的檔**（`.o` 檔案），如果涉及組合語言的話。
3. `ld` 連結器將多個目的檔或庫檔案（靜態或動態）連結成最終的 **可執行檔** 或 **動態函式庫**。
4. `make` 根據 `Makefile` 自動管理專案的編譯過程，生成所需的**目的檔**和**可執行檔**。



IPSI Lab, Yuntech-CSIE

如何使用 GNU 工具

- 在 Linux 當中
  - 預設就包含了 GNU 工具
- 在 MS. Windows 中
  - Mysys2（本課程以此為例）
  - WSL（Windows Subsystem for Linux）
  - Git for Windows
  - others

IPSI Lab, Yuntech-CSIE

習題

1. ~~請從網路下載Dev C++ 軟體，並參照附錄 D 的說明，安裝並使用 Dev C++ 撰寫 C 語言程式，並學習該軟體的用法。~~
2. ~~請找出 Dev C++ 當中的GNU工具，並在設定好 PATH 環境變數後，試用 gcc 指令編譯任意一個 C 語言程式 (設定方法請參考本書附錄 D)。~~

❖ **第三章將補充新的方法- MSYS2。**

IPSI Lab, Yuntech-CSIE

~~Windows上gcc編譯環境設定~~

1. ~~在C槽內找到Dev-C++內的bin資料夾，並複製其地址(不同電腦設置會有差異)~~
   - ~~C:\Program Files (x86)\Dev-Cpp\MinGW64\bin~~
   - ~~C:\Dev-Cpp\bin~~
2. ~~本機>右鍵選擇內容~~
3. ~~進階系統設定, 進階>環境變數~~
4. ~~系統變數>Path>新增~~
5. ~~新增第一步複製的地址~~
6. ~~成功！~~

IPSI Lab, Yuntech-CSIE

C 語言程式範例

▶範例 1.2 程式 main.c 與 sum.c

C 語言主程式 (main.c)

```
#include <stdio.h>
int main(void) {
      int sum1 = sum(10);
      printf("sum=%d\n", sum1);
      system("pause");
      return 1;
}
```

C 語言函數 (sum.c)

```
int sum(int n) {
      int s=0;
      int i;
      for (i=1; i<=n;i++) {
            s = s + i;
      }
      return s;
}
```

IPSI Lab, Yuntech-CSIE

使用 gcc 編譯 C 語言程式

圖 1.5　利用 gcc 同時編譯 main.c 與 sum.c 並輸出執行檔

```
C:\ch01>gcc sum.c main.c -o sum

C:\ch01>gcc sum.c main.c -o sum

C:\ch01>dir *.exe
 磁碟區 C 中的磁碟沒有標籤。
 磁碟區序號:  70AE-6E8A

 C:\ch01 的目錄

2010/03/12  上午 09:01                16,019 sum.exe
               1 個檔案                16,019 位元組
               0 個目錄  11,429,384,192 位元組可用

C:\ch01>sum
sum=55
請按任意鍵繼續 . . .
```

IPSI Lab, Yuntech-CSIE

使用 gcc 產生組合語言

- 使用 -S 參數可要求 gcc 產生組合語言

```
gcc -S sum.c -o sum.s
gcc -S main.c -o main.s
```

IPSI Lab, Yuntech-CSIE

將 gcc 當成組譯器使用

圖 1.6 將 gcc 當成組譯器使用

```
C:\ch01>gcc -S sum.c -o sum.s

C:\ch01>gcc -S main.c -o main.s

C:\ch01>gcc main.s sum.s -o sum2

C:\ch01>sum2
sum=55
請按任意鍵繼續 . . .
```

IPSI Lab, Yuntech-CSIE

同時組譯並連結

圖 1.7 利用 gcc 編譯 C 語言 main.c 同時組譯組合語言 sum.s

```
C:\ch01>gcc main.c sum.s -o sum3

C:\ch01>sum3
sum=55
請按任意鍵繼續 . . .
```

IPSI Lab, yuntech-CSIE

習題

1. 之前寫 C 程式是在哪一個作業系統？ 哪一個 IDE？（例：Dev-C++、Code::Blocks、VS、Xcode、VS Code、其他）
2. IDE 背後實際呼叫的是哪個編譯器？（例：MinGW gcc、TDM-GCC、MSVC cl.exe、Clang、Linux gcc、其他）
3. 使用 Command Line的方式呼叫編譯器進行範例程式編譯，以截圖呈現作答。

IPSI Lab, Yuntech-CSIE